

A Simple, Extensible Queue Migration Plan for Honcho

Phil Bascom · philbas.com · Platform Engineering take-home · 2026-06-27 · github.com/philosaether/pl-takehome-technical

This year, the Honcho application saw an orders-of-magnitude explosion in use, from ~ 200 active users to $\sim 30,000$. While its existing architecture has held up admirably, operational maturity will require a deliberate choice of implementation methodology.

In this paper, we propose a simple migration plan, with an optional second stage, and discuss circumstances in which the further migration is advisable. The first stage requires a minimal, ~ 150 -line alteration to the existing Postgres queue code, with no new infrastructure. We expect this change to yield a $\sim 1000\times$ faster scheduling look-ahead, sufficient to support another orders-of-magnitude growth cycle.

The second stage would move some or all work to a Valkey-backed in-memory queue. While it increases operational complexity and the risk of data loss, we show that Valkey offers up to $15\times$ the throughput of Postgres under certain conditions.

1 Low-hanging fruit is the most nutritious

By design, the Honcho driver processes representation work units only once they have crossed a threshold of pending cost. To day that value is recomputed every second by summing message token counts across the queue, grouped by work unit — an $O(N)$ scan that grows with queue depth, reaching ~ 2.5 s at 10^7 pending tasks (Fig. 1).

The remedy is to stop recomputing the answer and start maintaining it. We keep each work unit's pending cost in a small aggregate table, updated incrementally on enqueue and acknowledgement, and gate eligibility with a partial index the claim query reads directly. The scan becomes bounded by the number of *eligible* work units — a flat, small cost from 10^4 to 10^7 pending tasks. The change is bounded, surgical, and shippable in days (Fig. 2).

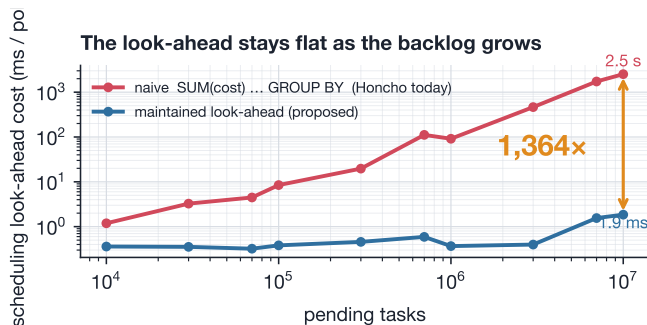


Fig. 1. The maintained look-ahead stays ~ 1 ms; the naive scan reaches 2.5 s at 10^7 tasks — a $1,364\times$ gap.

Fig. 2 — the change, against Honcho's real code (queue_manager.py#L347--361):

```
# NOW: O(unprocessed tasks), recomputed every poll
select (QueueItem.work_unit_key,
       func.sum(Message.token_count))
  .join(Message).where(~QueueItem.processed)
  .group_by(QueueItem.work_unit_key) # scan+join

# PROPOSED: O(eligible units), one indexed lookup
select (WorkUnit.work_unit_key)
  .where(WorkUnit.eligible)           # partial index
  .order_by(WorkUnit.oldest_pending_at)
  .with_for_update(skip_locked=True)
```

2 Efficiency is (sometimes) worth the effort

While Postgres-as-queue is a durable pattern — committed work survives a crash via write-ahead logging at the OS layer — Postgres is optimised for transactional integrity, not raw throughput. A dedicated hot-queue system such as Valkey offers significantly higher performance under load. That performance comes at a cost: greater operational complexity, and a risk of data loss through the fsync window.

Whether Honcho should adopt a Valkey queue is a nuanced question. When tasks are light-weight and workers execute quickly, Valkey offers a $15\times$ throughput increase per primary, and as much as $26\times$ at equal shard counts under peak load. But Valkey's lead shrinks to near-parity ($\sim 1.1\times$) as per-task time-to-process reaches 200 ms (Fig. 3).

We advise holding a Valkey migration until production data justifies it. To support the possibility, average time-to-process and peak worker count should be tracked as soon as possible.

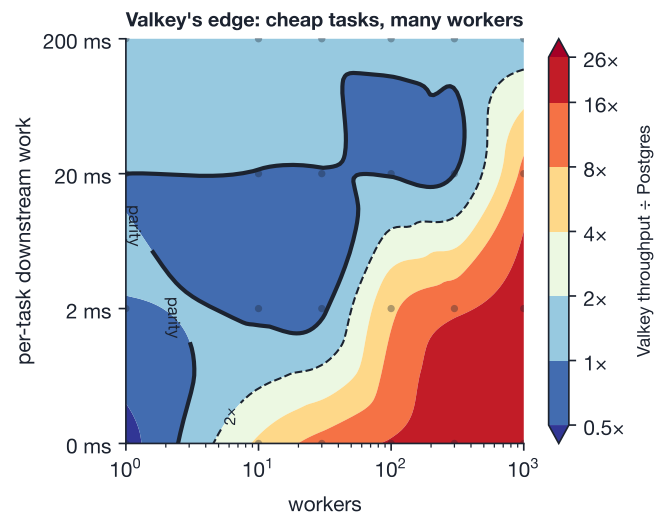


Fig. 3. Valkey's throughput advantage over Postgres across worker count and per-task work. The edge is a canyon for cheap, high-concurrency work and collapses to parity at LLM-scale latency.

3 Methods

The look-ahead test ran two near-identical queues differing only in a single SQL query, isolating the variance to algorithmic complexity, locally in a Docker container on a 2.6 GHz MacBook Pro (reproduce under `tests/bench`). The head-to-head ran two per-backend Docker images implementing one shared queue contract against a common load generator on AWS (`deploy/terraform`); a Zipfian key distribution drove continuous work-unit churn, and correctness was established deterministically by the shared `tests/conformance` package. Figure 3 interpolates four per-task work levels {0, 2, 20, 200 ms} across worker counts from 1 to 1,000; producer-bound Postgres points are conservative lower bounds. The full experiment ran in ~ 90 minutes for under \$10.

Reproducibility: make load-test reproduces the numbers; environment pinned. **Fairness:** a per-tenant in-flight cap plus weighted claim order is the one-seam extension (designed, not built). **Flush policy:** a per-head age cap — every task is promoted when it becomes the oldest and ages out, so one slow task cannot strand the work behind it. **Valkey runs on less iron:** each Valkey primary ran on the *same* box class as Postgres though, single-threaded, it cannot use it — so the gap is a conservative floor. **Stage-2 hardening:** a standalone reaper, versioned migrations, a managed durable Valkey tier. **Sharpening Fig. 3:** 5/50/100 ms work-points and the durability curve would resolve the crossover precisely.